

Etapa 1 – Modelado de la base de datos en MongoDB

Colecciones implementadas

1. restaurante

- Contiene datos del restaurante: nombre, dirección, teléfono y categoría.
- Usada como referencia en varias otras colecciones (menú, orden, reseña).
- **Usos y consultas esperadas**
 - Identificar usuarios registrados.
 - Obtener restaurantes por nombre o categoría.
 - Generar reportes de los restaurantes mejor calificados (agregación con resena).
 - Buscar rápidamente un restaurante por texto.
- **Índices:**
 1. **Índice simple:**

```
db.restaurante.createIndex({ nombre: 1 })
```

Búsqueda rápida por nombre del restaurante.
 2. **Índice de texto:**

```
db.restaurante.createIndex({ nombre: "text", categoria: "text" })
```

Para búsquedas de texto en nombre y categoría.

2. usuario

- Almacena los datos del cliente: nombre, email, dirección, teléfono, contraseña, y fecha de registro.
- Relacionado con las colecciones de orden y reseña.
- **Usos y consultas esperadas**
 - Identificar usuarios registrados.
 - Obtener actividad de un usuario (órdenes, reseñas).
- **Índices:**
 1. **Índice compuesto:**

```
db.usuario.createIndex({ nombre: 1, direccion: 1 })
```

Mejora búsquedas por nombre y ubicación.

3. articulo_menu

- Representa los productos del menú ofrecidos por los restaurantes.
- Incluye nombre, precio, descripción, disponibilidad y restaurante_id referenciado.
- **Usos y consultas esperadas**
 - Listar el menú de un restaurante.
 - Filtrar por disponibilidad.
 - Búsquedas por nombre o descripción del platillo.

- **Índices:**

1. **Índice compuesto:**

```
db.menu.createIndex({ restaurante_id: 1, disponible: 1 })
```

Para listar artículos disponibles por restaurante.

2. **Índice de texto:**

```
db.menu.createIndex({ nombre: "text", descripcion: "text" })
```

Permite búsquedas por nombre y descripción del platillo.

4. **orden**

- Documento que incluye usuario, restaurante, fecha, estado del pedido y un array de platillos.
- Cada platillo contiene menu_item_id, cantidad y precio unitario → **documento embebido** dentro de la orden.

- **Usos y consultas esperadas**

- Identificar usuarios registrados.
- Listar pedidos por usuario y fecha.
- Consultar estado del pedido.
- Reportes de platillos más vendidos (con \$unwind + \$group).
- Consultas por rango de fechas, estados, o totales.

- **Índices:**

1. **Índice compuesto:**

```
db.orden.createIndex({ usuario_id: 1, fecha: -1 })
```

Consultas por órdenes recientes de un usuario.

2. **Índice multikey (por array):**

```
db.orden.createIndex({ "platillos.menu_item_id": 1 })
```

Mejora consultas por items de menú en pedidos.

5. **resena**

- Contiene calificación y comentario que el usuario deja para un restaurante, vinculado además a una orden específica.
- Relación referenciada a usuario, orden y restaurante

- **Usos y consultas esperadas**

- Ver reseñas de un restaurante.
- Obtener calificación promedio.
- Ver comentarios hechos por un usuario.
- Agregaciones por calificación y fecha.

- **Índices:**

1. **Índice compuesto:**

```
db.resena.createIndex({ restaurante_id: 1, calificacion: -1 })
```

Consultas por calificaciones de un restaurante.

2. **Índice simple:**

```
db.resena.createIndex({ usuario_id: 1 })
```

Para ver reseñas hechas por un usuario.

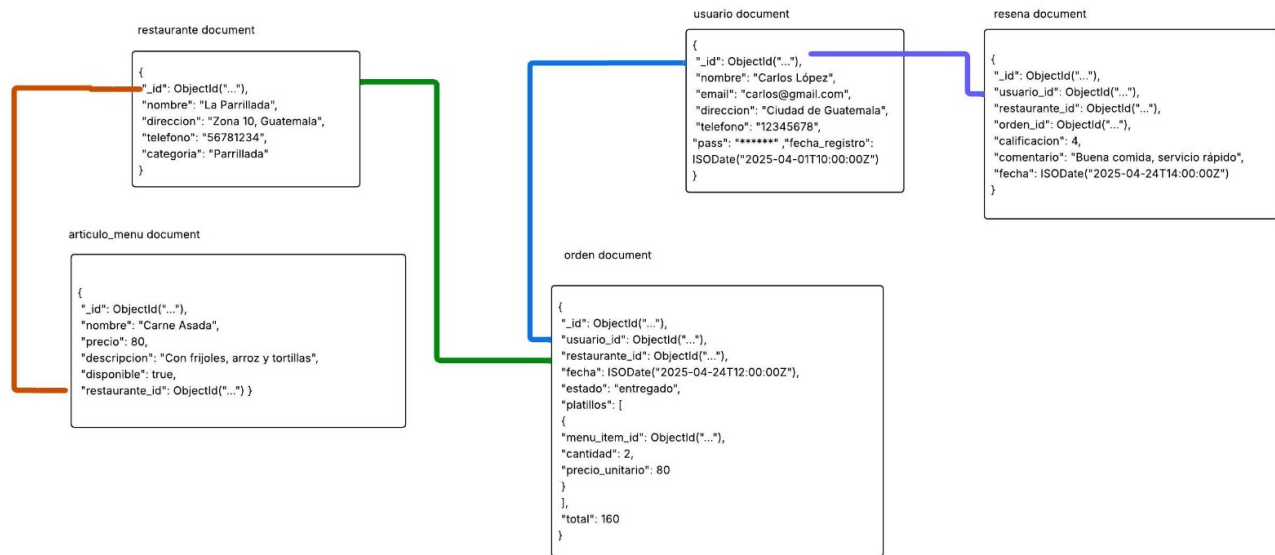


Figure 1: Diagrama de Modelo de Datos NoSQL

Justificación de estructuras embebidas vs referenciadas

- Se **embeben los platillos** dentro de la orden para facilitar consultas frecuentes como el detalle completo del pedido.
- Se **referencian usuarios, restaurantes, artículos y órdenes** para evitar redundancia y facilitar mantenibilidad (por ejemplo, si un nombre de restaurante cambia, no hay que actualizar múltiples órdenes).

Precarga de datos (mongoimport)

Usando la sintaxis mencionada en el **siguiente enlace**.

Se puede hacer el siguiente comando para poblar la base de datos con los archivos json definidos en el repositorio:

```
mongoimport --uri "<uri>" --collection "<nombre_coleccion>" --file
↳ "<ruta_al_archivo_de_importacion>"
```

```
mongoimport --uri "uri" --collection usuario --file "./data/usuarios.json"
↳ --jsonArray
```

```
mongoimport --uri "uri" --collection restaurante --file
↳ "./data/restaurantes.json" --jsonArray
```

```
mongoimport --uri "uri" --collection menu --file "./data/articulos_menu.json"
↳ --jsonArray
```

```

mongoimport --uri "uri" --collection resena --file "./data/resenas.json"
↪ --jsonArray
mongoimport --uri "uri" --collection orden --file "./data/ordenes_0.json"
↪ --jsonArray
mongoimport --uri "uri" --collection orden --file "./data/ordenes_1.json"
↪ --jsonArray
mongoimport --uri "uri" --collection orden --file "./data/ordenes_2.json"
↪ --jsonArray
mongoimport --uri "uri" --collection orden --file "./data/ordenes_3.json"
↪ --jsonArray
mongoimport --uri "uri" --collection orden --file "./data/ordenes_4.json"
↪ --jsonArray

```

Política para evitar consultas sin índices

Conforme a los requerimientos del proyecto, la base de datos MongoDB debe estar diseñada para **rechazar o prevenir consultas que no utilicen índices**, garantizando una ejecución eficiente y escalable. Para lograrlo, se aplican las siguientes medidas:

Diseño basado en índices

- Todas las colecciones principales (restaurante, usuario, menu, orden, resena) cuentan con **al menos dos índices adecuados** (simples, compuestos, multikey o de texto).
- Los índices fueron planificados con base en las futuras consultas REST que se implementarán en la API, considerando filtros, ordenamientos y búsquedas.

Validación con `explain("executionStats")`

Antes de incorporar una consulta a la API, se valida con:

```
db.coleccion.find({ campo: valor }).explain("executionStats")
```

Este comando permite verificar el plan de ejecución y asegurarse de que la consulta usa IXSCAN (lectura por índice) y no COLLSCAN (lectura completa de la colección).

Uso de `.hint()` en MongoDB

La función `.hint()` permite **forzar explícitamente el uso de un índice específico** durante una consulta. Esto se utiliza para:

- Asegurar que una consulta use un índice existente.

- Forzar un error si se proporciona un índice inexistente o incompatible, ayudando a validar el diseño.

Sintaxis

```
db.coleccion.find(filtro, proyeccion).hint({ campo1: 1, campo2: -1 })
```

- campo1, campo2, etc.: deben coincidir exactamente con el índice existente.
- Los valores 1 o -1 indican el orden (ascendente o descendente) del índice.
- Si el índice especificado no existe o no es compatible, la consulta fallará.

Ejemplo correcto (índice existente)

```
db.resena.find(  
  { restaurante_id: "22e49f7d-c7d9-42c0-9785-1507f2003c19" },  
  { calificacion: 1, comentario: 1, fecha: 1, _id: 0 }  
)  
.hint({ restaurante_id: 1, calificacion: -1 })  
.limit(5)  
.explain("executionStats")
```

Esta consulta utiliza correctamente el índice `restaurante_id_1_calificacion_-1` y es eficiente.

Ejemplo incorrecto (índice inexistente)

```
db.resena.find(  
  { restaurante_id: "22e49f7d-c7d9-42c0-9785-1507f2003c19" }  
)  
.hint({ fecha: -1 }) // Este índice no está definido  
.limit(5)
```

Resultado esperado:

MongoServerError: hint provided does not correspond to an existing index

Este comportamiento confirma que la base de datos no ejecutará consultas si no pueden resolverse mediante índices definidos, cumpliendo con la política de eficiencia.